House Prices Prediction Midterm Project

Name: Taylor McDonald

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

Step 2: Load the Data

We only use train.csv for this project. We split it ourselves into a training set and a validation set so we can test the models on data they haven't seen.

```
df = pd.read_csv('data/train.csv')
print('Shape:', df.shape)
df.head()
```

```
Shape: (1460, 81)
```

```
   Id  MSSubClass MSZoning  LotFrontage  LotArea Street Alley LotShape
\
0   1          60       RL         65.0     8450   Pave   NaN      Reg

1   2          20       RL         80.0     9600   Pave   NaN      Reg

2   3          60       RL         68.0    11250   Pave   NaN      IR1

3   4          70       RL         60.0     9550   Pave   NaN      IR1

4   5          60       RL         84.0    14260   Pave   NaN      IR1


  LandContour Utilities  ... PoolArea PoolQC Fence MiscFeature MiscVal
MoSold  \
0         Lvl    AllPub  ...        0    NaN   NaN         NaN       0
2
1         Lvl    AllPub  ...        0    NaN   NaN         NaN       0
```

```
5
2         Lvl    AllPub  ...        0    NaN   NaN         NaN       0
9
3         Lvl    AllPub  ...        0    NaN   NaN         NaN       0
2
4         Lvl    AllPub  ...        0    NaN   NaN         NaN       0
12

  YrSold  SaleType  SaleCondition  SalePrice
0   2008        WD         Normal     208500
1   2007        WD         Normal     181500
2   2008        WD         Normal     223500
3   2006        WD        Abnorml     140000
4   2008        WD         Normal     250000

[5 rows x 81 columns]
```

```
df.describe()
```

```
                Id   MSSubClass  LotFrontage        LotArea
OverallQual  \
count  1460.000000  1460.000000  1201.000000    1460.000000
1460.000000
mean    730.500000    56.897260    70.049958   10516.828082
6.099315
std     421.610009    42.300571    24.284752    9981.264932
1.382997
min       1.000000    20.000000    21.000000    1300.000000
1.000000
25%     365.750000    20.000000    59.000000    7553.500000
5.000000
50%     730.500000    50.000000    69.000000    9478.500000
6.000000
75%    1095.250000    70.000000    80.000000   11601.500000
7.000000
max    1460.000000   190.000000   313.000000  215245.000000
10.000000

       OverallCond    YearBuilt  YearRemodAdd   MasVnrArea
BsmtFinSF1  ...  \
count  1460.000000  1460.000000   1460.000000  1452.000000
1460.000000  ...
mean      5.575342  1971.267808   1984.865753   103.685262
443.639726  ...
std       1.112799    30.202904     20.645407   181.066207
456.098091  ...
min       1.000000  1872.000000   1950.000000     0.000000
0.000000  ...
25%       5.000000  1954.000000   1967.000000     0.000000
0.000000  ...
```

```
50%       5.000000  1973.000000   1994.000000     0.000000
383.500000  ...
75%       6.000000  2000.000000   2004.000000   166.000000
712.250000  ...
max       9.000000  2010.000000   2010.000000  1600.000000
5644.000000  ...

        WoodDeckSF  OpenPorchSF  EnclosedPorch    3SsnPorch
ScreenPorch  \
count  1460.000000  1460.000000    1460.000000  1460.000000
1460.000000
mean     94.244521    46.660274      21.954110     3.409589
15.060959
std     125.338794    66.256028      61.119149    29.317331
55.757415
min       0.000000     0.000000       0.000000     0.000000
0.000000
25%       0.000000     0.000000       0.000000     0.000000
0.000000
50%       0.000000    25.000000       0.000000     0.000000
0.000000
75%     168.000000    68.000000       0.000000     0.000000
0.000000
max     857.000000   547.000000     552.000000   508.000000
480.000000

          PoolArea       MiscVal       MoSold       YrSold
SalePrice
count  1460.000000   1460.000000  1460.000000  1460.000000
1460.000000
mean      2.758904     43.489041     6.321918  2007.815753
180921.195890
std      40.177307    496.123024     2.703626     1.328095
79442.502883
min       0.000000      0.000000     1.000000  2006.000000
34900.000000
25%       0.000000      0.000000     5.000000  2007.000000
129975.000000
50%       0.000000      0.000000     6.000000  2008.000000
163000.000000
75%       0.000000      0.000000     8.000000  2009.000000
214000.000000
max     738.000000  15500.000000    12.000000  2010.000000
755000.000000

[8 rows x 38 columns]
```

```
# Check data types
print(df.dtypes.value_counts())
```

```
object     43
int64      35
float64     3
Name: count, dtype: int64
```

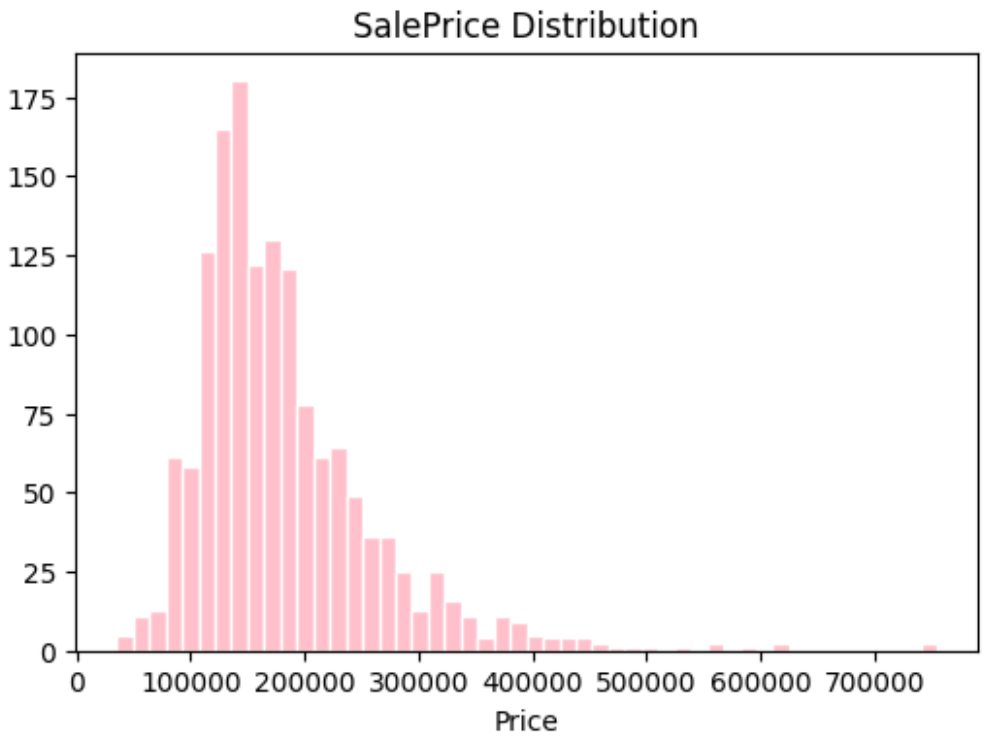
Step 3: Data Analysis

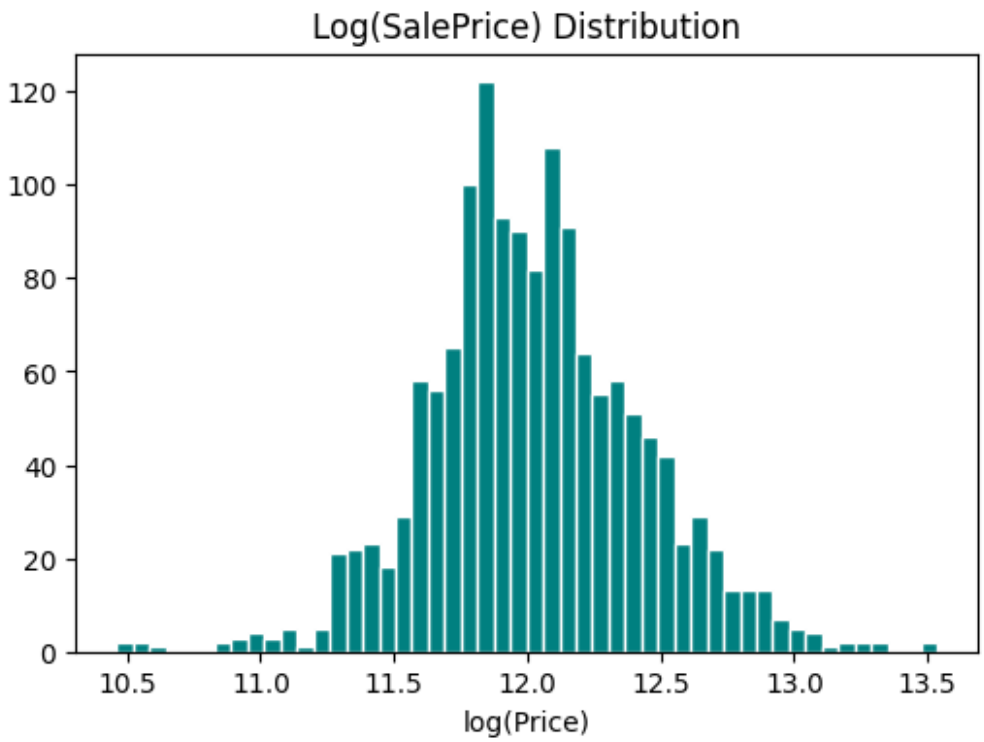
Before building any model, let's look at the data to understand what were working with. This step helps us spot patterns, weird values, and which features might matter most.

3.1 Looking at SalePrice

First let's look at the distribution of house prices. We'll see that it's skewed to the right (a few very expensive houses pull the average up). Taking the log makes it look more like a bell curve, which helps the models learn better.

```
# Raw price distribution
plt.figure(figsize=(6, 4))
plt.hist(df['SalePrice'], bins=50, color='pink', edgecolor='white')
plt.title('SalePrice Distribution')
plt.xlabel('Price')
plt.show()

# Log transformed
plt.figure(figsize=(6, 4))
plt.hist(np.log1p(df['SalePrice']), bins=50, color='teal',
edgecolor='white')
plt.title('Log(SalePrice) Distribution')
plt.xlabel('log(Price)')
plt.show()

print('Skewness (original):', round(df['SalePrice'].skew(), 3))
print('Skewness (log):', round(np.log1p(df['SalePrice']).skew(), 3))
print('The log-transformed version looks more normal, so we will
predict log(SalePrice)')
```



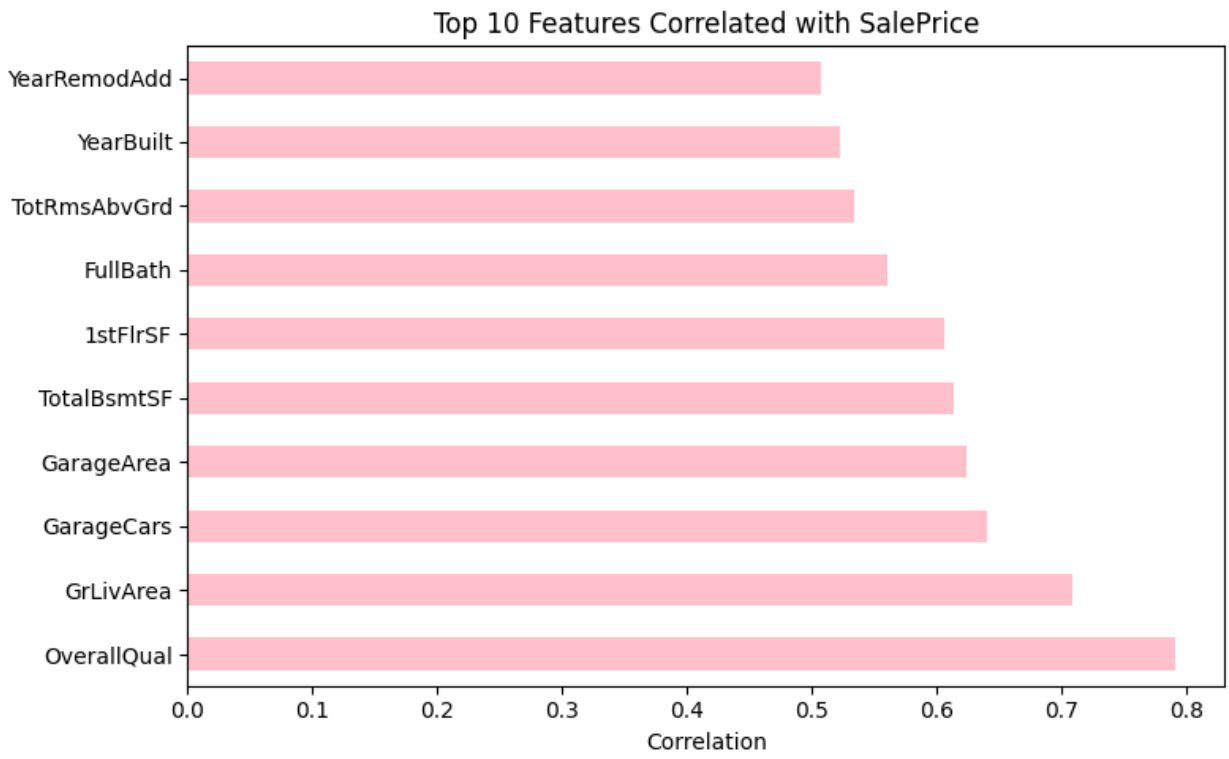


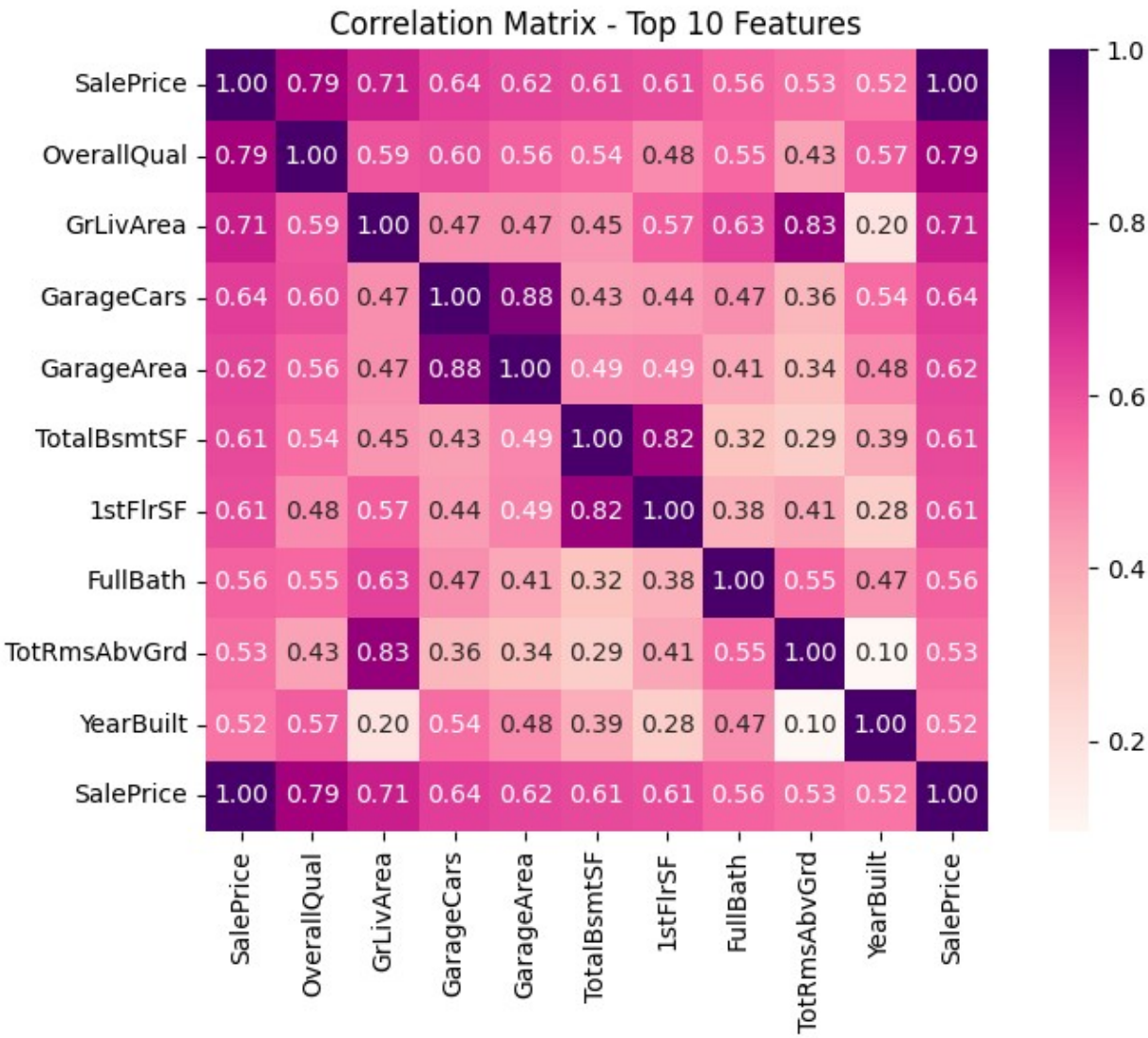

```
Skewness (original): 1.883
Skewness (log): 0.121
```

```
The log-transformed version looks more normal, so we will predict
log(SalePrice)
```

3.2 What features are most correlated with SalePrice?

```
# Top 10 numeric features correlated with SalePrice
corr = df.select_dtypes(include=np.number).corr()
['SalePrice'].sort_values(ascending=False)

# Bar chart
plt.figure(figsize=(8, 5))
corr[1:11].plot(kind='barh', color='pink')
plt.title('Top 10 Features Correlated with SalePrice')
plt.xlabel('Correlation')
plt.tight_layout()
plt.show()

# Heatmap
top10_cols = corr.head(10).index.tolist() + ['SalePrice']
plt.figure(figsize=(8, 6))
sns.heatmap(df[top10_cols].corr(), annot=True, fmt='.2f', cmap='RdPu',
square=True)
plt.title('Correlation Matrix - Top 10 Features')
plt.tight_layout()
plt.show()

print(corr[1:11])
```

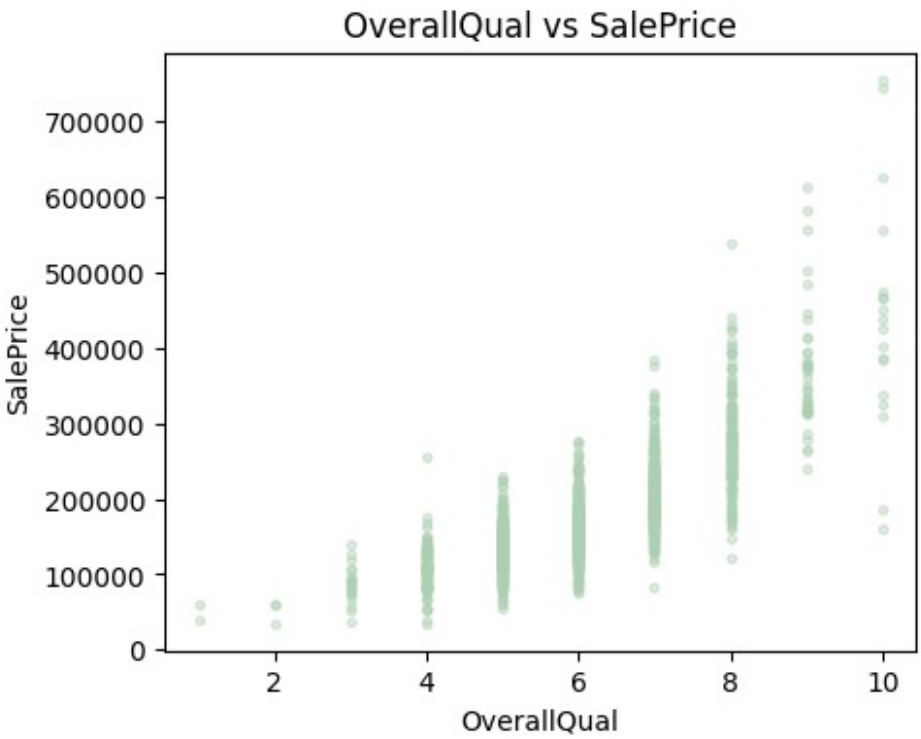

Top 10 Features Correlated with SalePrice
YearRemodAdd
YearBuilt
TotRmsAbvGrd
FullBath
1stFlrSF
TotalBsmtSF
GarageArea
GarageCars
GrLivArea
OverallQual
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
Correlation

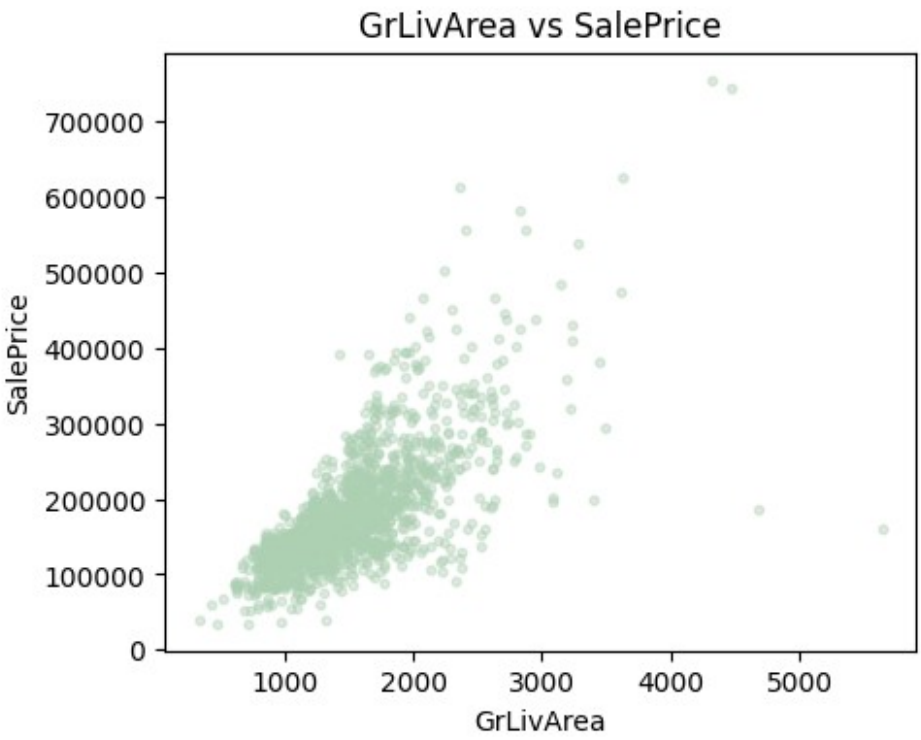


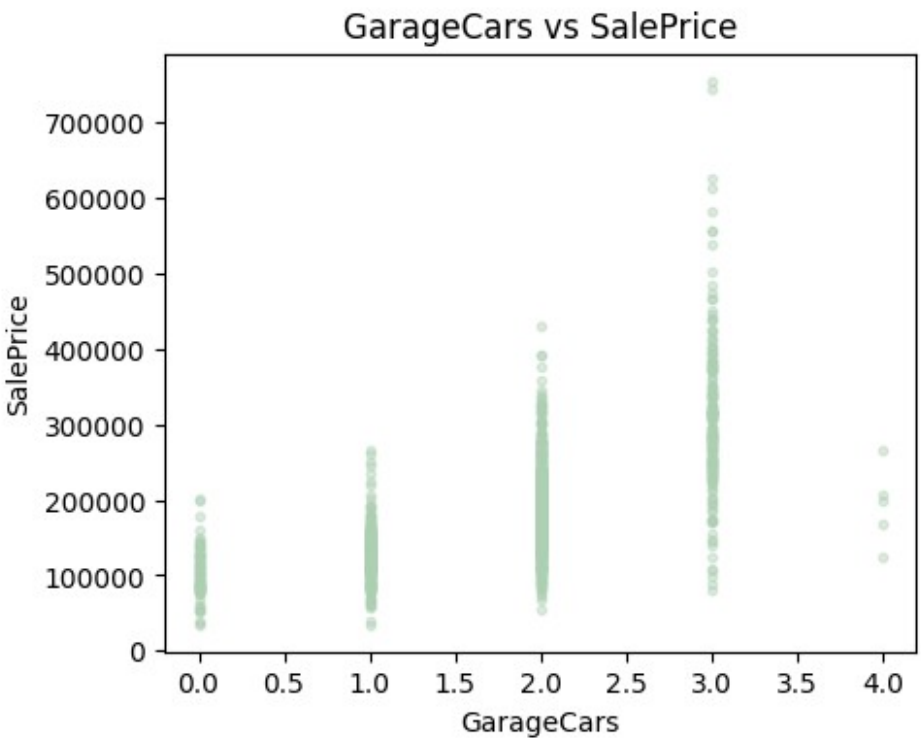

```
OverallQual     0.790982
GrLivArea       0.708624
GarageCars      0.640409
GarageArea      0.623431
TotalBsmtSF     0.613581
1stFlrSF        0.605852
FullBath        0.560664
TotRmsAbvGrd    0.533723
YearBuilt       0.522897
YearRemodAdd    0.507101
Name: SalePrice, dtype: float64
```

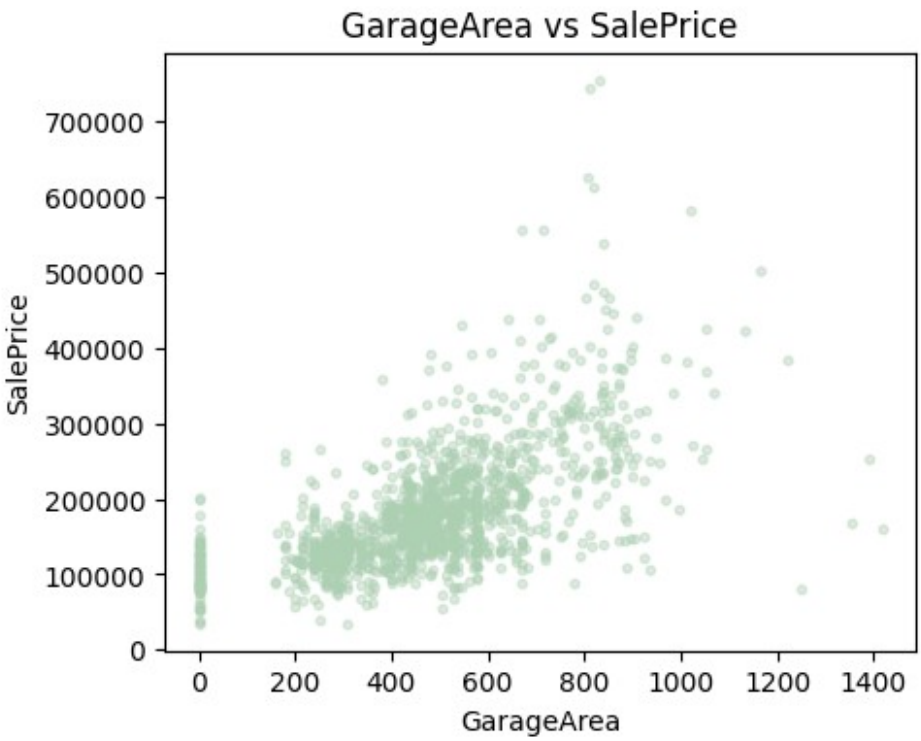
```
# Scatter plot of top 4 features vs SalePrice
top4 = corr[1:5].index.tolist()

for col in top4:
```

```
    plt.figure(figsize=(5, 4))
    plt.scatter(df[col], df['SalePrice'], alpha=0.4,
s=10,color='#ADD0B3')
    plt.xlabel(col)
    plt.ylabel('SalePrice')
    plt.title(f'{col} vs SalePrice')
    plt.show()
```



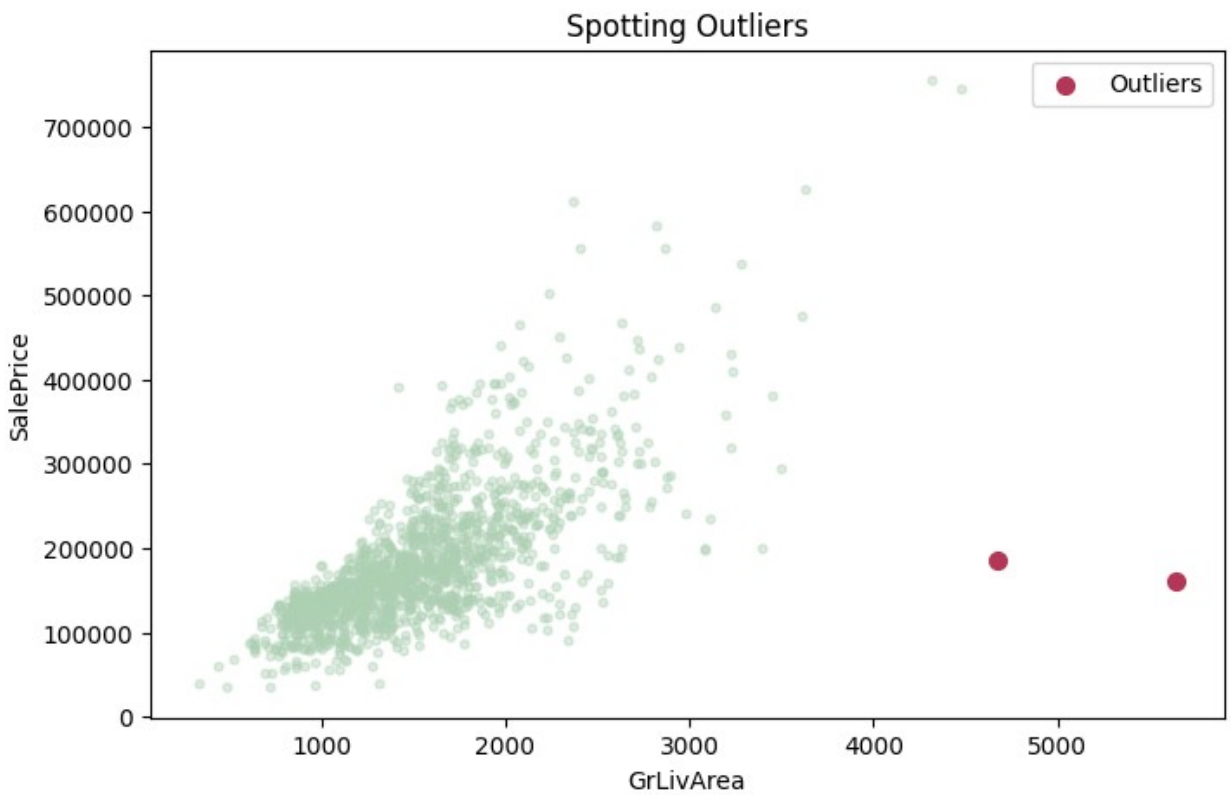

GrLivArea vs SalePrice
SalePrice
GrLivArea
0
100000
200000
300000
400000
500000
600000
700000
1000
2000
3000
4000
5000


GarageCars vs SalePrice
SalePrice
GarageCars
0
100000
200000
300000
400000
500000
600000
700000
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0



3.3 Removing Outliers

```
# Two houses with very large area but very low price, likely data
errors
plt.figure(figsize=(8, 5))
plt.scatter(df['GrLivArea'], df['SalePrice'], alpha=0.4,
s=12,color='#ADD0B3')
plt.scatter(df[(df['GrLivArea'] > 4000) & (df['SalePrice'] < 300000)]
['GrLivArea'],
            df[(df['GrLivArea'] > 4000) & (df['SalePrice'] < 300000)]
['SalePrice'],
            color='#B43757', s=50, label='Outliers')
plt.xlabel('GrLivArea')
plt.ylabel('SalePrice')
plt.title('Spotting Outliers')
plt.legend()
plt.show()

# Remove them
df = df[~((df['GrLivArea'] > 4000) & (df['SalePrice'] <
300000))].reset_index(drop=True)
print('Dataset size after removing outliers:', df.shape)
```



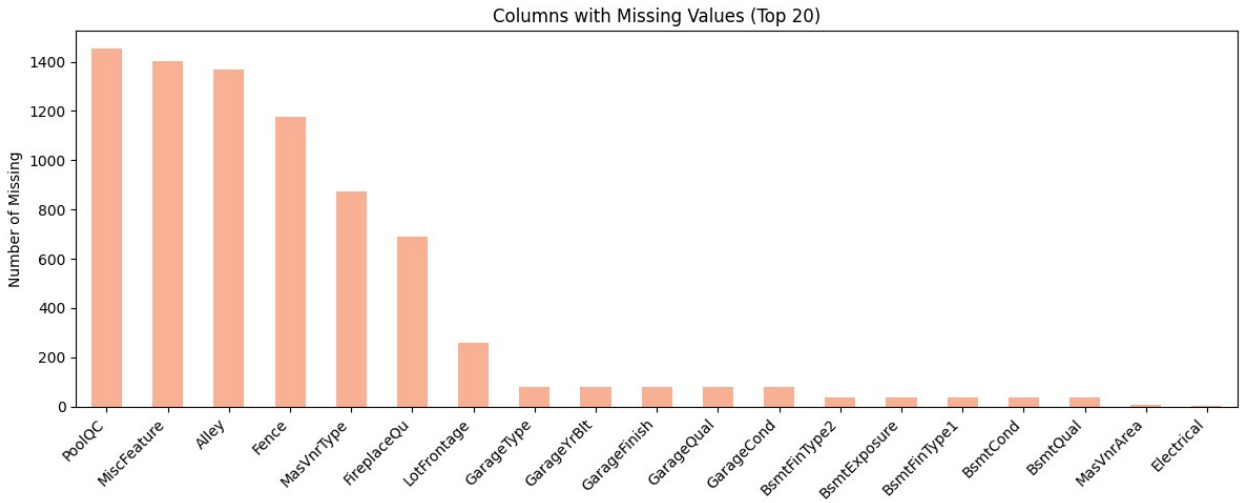

```
Dataset size after removing outliers: (1458, 81)
```

Step 4: Handling Missing Values

Most real datasets have missing values. Before we can train any model, we need to fill them in. The tricky part is figuring out why a value is missiing, sometimes it just means the house doesn't have that feature at all.

```
# How many missing values does each column have?
missing = df.isnull().sum()
missing = missing[missing > 0].sort_values(ascending=False)

if len(missing) == 0:
    print("No missing values found.")
else:
    plt.figure(figsize=(12, 5))
    missing.head(20).plot(kind='bar', color='#F8B195')
    plt.title('Columns with Missing Values (Top 20)')
    plt.ylabel('Number of Missing')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
```

```
print(missing.head(20))
```




```
PoolQC          1452
MiscFeature     1404
Alley           1367
Fence           1177
MasVnrType       872
FireplaceQu      690
LotFrontage      259
GarageType        81
GarageYrBlt       81
GarageFinish      81
GarageQual        81
GarageCond        81
BsmtFinType2      38
BsmtExposure      38
BsmtFinType1      37
BsmtCond          37
BsmtQual          37
MasVnrArea         8
Electrical         1
dtype: int64
```

For a lot of columns, NaN doesn't mean the data is unknown, it means the house **doesn't have that feature**. For example, if there's no garage, all the garage columns will be NaN. We fill those with 'None' for text columns and 0 for numbericak columns.

```
# These NaN values mean the feature simply doesn't exist on that
property
no_feature_str = ['PoolQC', 'MiscFeature', 'Alley', 'Fence',
'FireplaceQu',
                  'GarageType', 'GarageFinish', 'GarageQual',
```

```
'GarageCond',
                   'BsmtQual', 'BsmtCond', 'BsmtExposure',
'BsmtFinType1', 'BsmtFinType2',
                   'MasVnrType']
for col in no_feature_str:
    df[col] = df[col].fillna('None')

# Same but for numeric columns (no garage = 0 sqft, etc.)
no_feature_num = ['GarageYrBlt', 'GarageArea', 'GarageCars',
                   'BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF',
'TotalBsmtSF',
                   'BsmtFullBath', 'BsmtHalfBath', 'MasVnrArea']
for col in no_feature_num:
    df[col] = df[col].fillna(0)

# LotFrontage,  use the median of the neighborhood (similar streets)
df['LotFrontage'] = df.groupby('Neighborhood')
['LotFrontage'].transform(
    lambda x: x.fillna(x.median())
)

# Everything else,  fill with the most common value
for col in df.select_dtypes(include='object').columns:
    df[col] = df[col].fillna(df[col].mode()[0])

for col in df.select_dtypes(include=np.number).columns:
    df[col] = df[col].fillna(df[col].median())

print('Missing values remaining:', df.isnull().sum().sum())

Missing values remaining: 0
```

Step 5: Feature Engineering

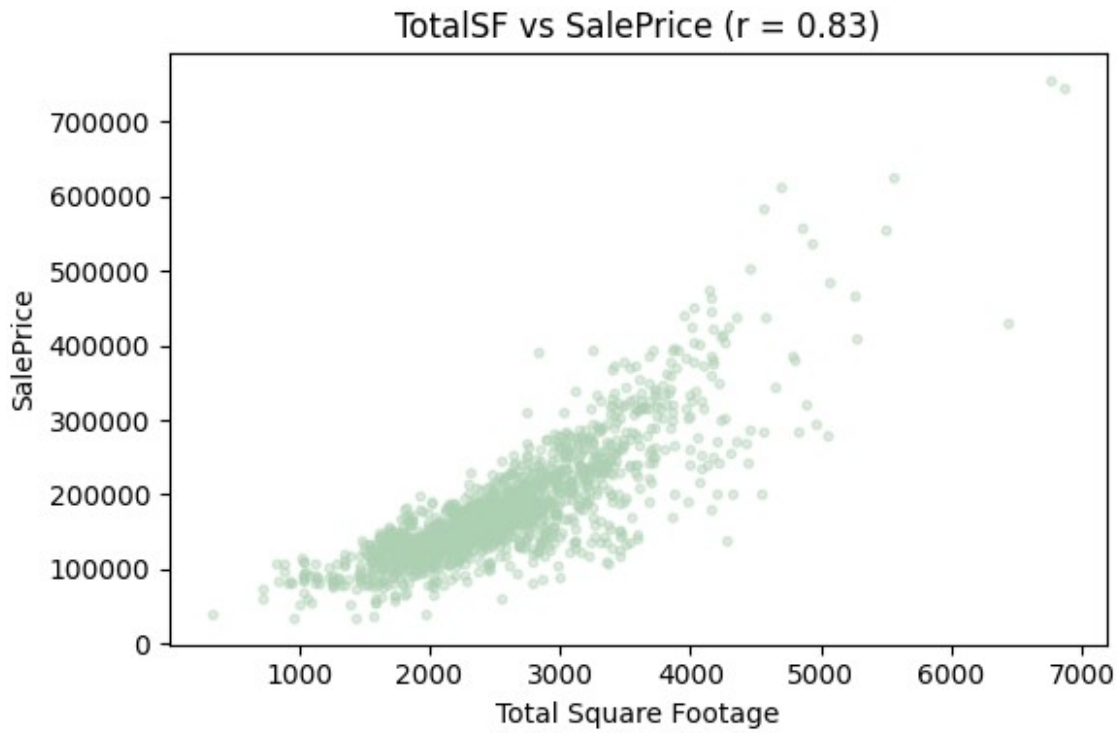
Feature engineering means creating new columns from the ones we already have. We can combine or transform existing features to give the model more useful information.

New Feature 1: TotalSF (Total Square Footage)

The dataset has three separate columns for area: basement, 1st floor, and 2nd floor. But really what matters is the total size of the house. We combine all three into one number. Bigger house = higher price, and now the model has one clear feature to work with instead of three separate ones.

```
df['TotalSF'] = df['TotalBsmtSF'] + df['1stFlrSF'] + df['2ndFlrSF']

plt.figure(figsize=(6, 4))
plt.scatter(df['TotalSF'], df['SalePrice'], alpha=0.4,
s=10,color="#ADD0B3")
```

```
plt.xlabel('Total Square Footage')
plt.ylabel('SalePrice')
plt.title(f'TotalSF vs SalePrice (r =
{df["TotalSF"].corr(df["SalePrice"]):.2f})')
plt.tight_layout()
plt.show()
```



New Feature 2: TotalBathrooms

The dataset splits bathrooms into 4 columns (full bath, half bath, basement full, basement half). Also, its good to note that a half bathroom is worth less than a full one, so we weight them at 0.5. This gives us one simple number that captures the total bathroom situation.

```
df['TotalBathrooms'] = (df['FullBath'] + df['BsmtFullBath'] +
                        0.5 * df['HalfBath'] + 0.5 *
df['BsmtHalfBath'])

print('TotalBathrooms value counts:')
print(df['TotalBathrooms'].value_counts().sort_index())
```

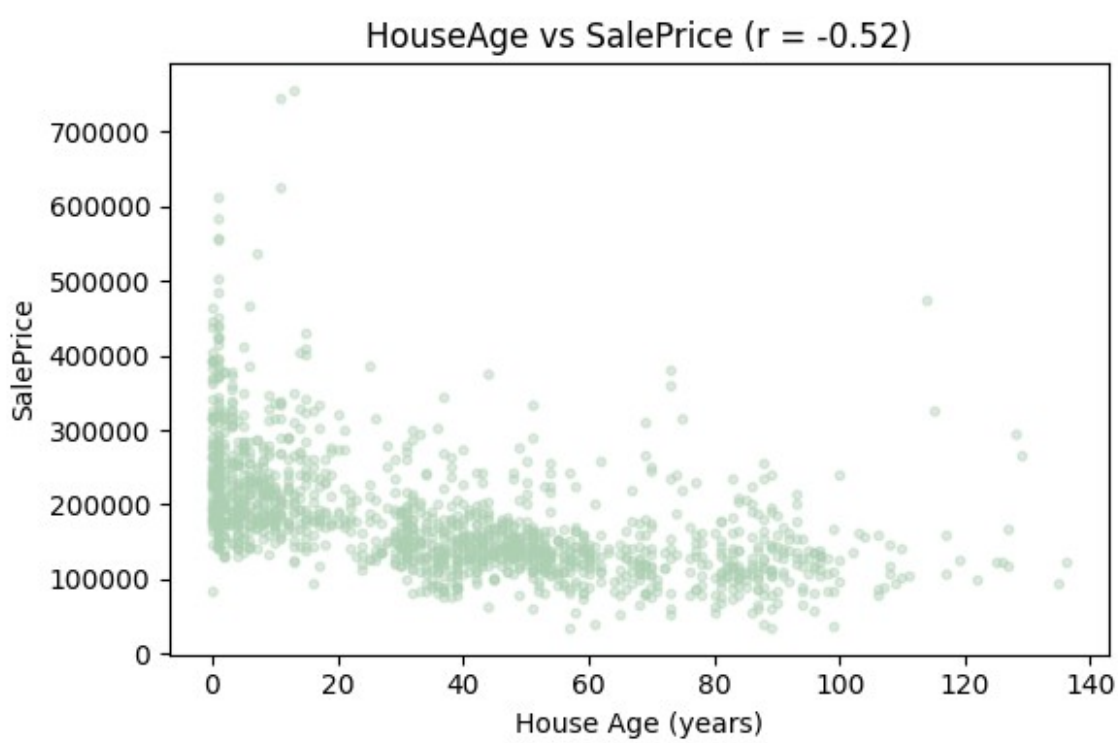
```
TotalBathrooms value counts:
TotalBathrooms
1.0    228
1.5    129
2.0    456
2.5    295
```

```
3.0    186
3.5    144
4.0     13
4.5      5
5.0      1
6.0      1
Name: count, dtype: int64
```

New Feature 3: HouseAge

The raw dataset gives us YearBuilt, but the actual year number isn't very meaningful to a model. What matters is how old the house is. A newer house is generally worth more, so we convert year built into age by subtracting from the year it was sold.

```
df['HouseAge'] = df['YrSold'] - df['YearBuilt']

plt.figure(figsize=(6, 4))
plt.scatter(df['HouseAge'], df['SalePrice'], alpha=0.4, s=10,
color='#ADD0B3')
plt.xlabel('House Age (years)')
plt.ylabel('SalePrice')
plt.title(f'HouseAge vs SalePrice (r =
{df["HouseAge"].corr(df["SalePrice"]):.2f})')
plt.tight_layout()
plt.show()
```



New Feature 4: OverallScore (Quality x Condition)

The dataset has OverallQual (quality of materials) and OverallCond (condition of the house) as separate columns. We multiply them together to get a combined score. A house that scores high on both should be worth a lot more than one that's average on each.

```
df['OverallScore'] = df['OverallQual'] * df['OverallCond']
print('OverallScore range:', df['OverallScore'].min(), '-',
df['OverallScore'].max())
```

```
OverallScore range: 1 - 90
```

New Feature 5: HasGarage, HasPool, Has2ndFloor

Sometimes just having something matters more than how big it is. We create simple yes/no flags (1 = has it, 0 = doesn't) for a garage, pool, and second floor. These let the model learn a simple "does it have this or not" effect.

```
df['HasGarage']   = (df['GarageArea'] > 0).astype(int)
df['HasPool']     = (df['PoolArea'] > 0).astype(int)
df['Has2ndFloor'] = (df['2ndFlrSF'] > 0).astype(int)

print('Homes with garage:', df['HasGarage'].sum())
print('Homes with pool:  ', df['HasPool'].sum())
print('Homes with 2nd floor:', df['Has2ndFloor'].sum())
```

```
Homes with garage: 1377
Homes with pool:   6
Homes with 2nd floor: 629
```

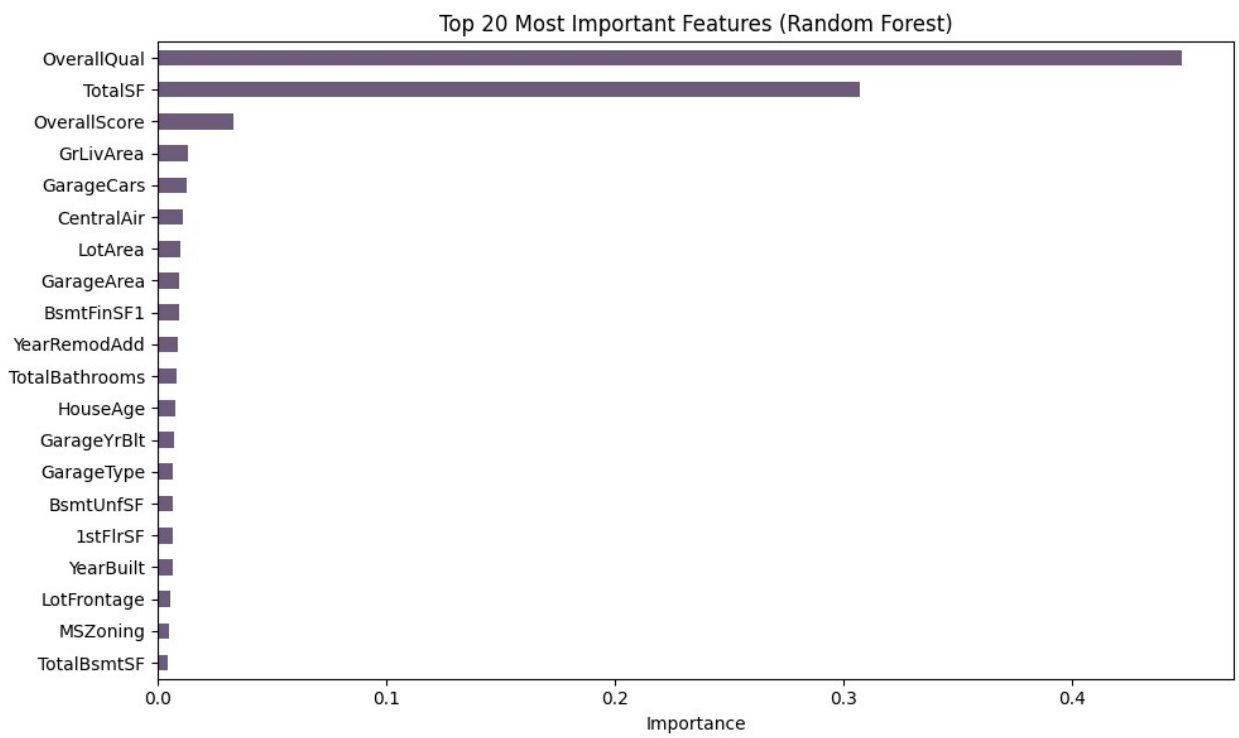
Step 6: Feature Selection

After adding the new features we now have over 80 columns to choose form. Using all of them can actually hurt performance because some features are noisy or irrelevant. We use a Random Forest to rank every feature by how useful it is, then keep only the top 10.

```
# First encode all categorical columns to numbers
df_model = df.drop(['Id', 'SalePrice'], axis=1).copy()
y = np.log1p(df['SalePrice'])

le = LabelEncoder()
for col in df_model.select_dtypes(include='object').columns:
    df_model[col] = le.fit_transform(df_model[col].astype(str))

print('Total features:', df_model.shape[1])
```

```
Total features: 86
```

```python
# Fit a random forest just to get feature importances
rf_temp = RandomForestRegressor(n_estimators=100, random_state=42)
rf_temp.fit(df_model, y)

importances = pd.Series(rf_temp.feature_importances_,
index=df_model.columns)
importances = importances.sort_values(ascending=False)

plt.figure(figsize=(10, 6))
importances.head(20).plot(kind='barh', color='#6C5B7B')
plt.gca().invert_yaxis()
plt.title('Top 20 Most Important Features (Random Forest)')
plt.xlabel('Importance')
plt.tight_layout()
plt.show()
```




```python
# Pick the top 10 features
top10 = importances.head(10).index.tolist()
print('Top 10 features selected:')
for i, feat in enumerate(top10, 1):
    print(f'  {i}. {feat}')
```

```
Top 10 features selected:
  1. OverallQual
  2. TotalSF
  3. OverallScore
  4. GrLivArea
```

```
  5. GarageCars
  6. CentralAir
  7. LotArea
  8. GarageArea
  9. BsmtFinSF1
  10. YearRemodAdd
```

Step 7: Train/Validation Split

We split the dataset into two parts:

- **Training set (80%)**: the model learns from this
- **Validation set (20%)**: we use this to test how well it learned

The key is: the model never sees the validation set during training. This gives us a honest measure of how it would perform on new houses.

```
X = df_model[top10]

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
random_state=42)

print('Training samples:  ', len(X_train))
print('Validation samples:', len(X_val))
```

```
Training samples:   1166
Validation samples: 292
```

```
# Scale the features ( important for Linear Regression and KNN)
# We fit the scaler on training data only, never on validation data!
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_val_sc   = scaler.transform(X_val)
```

Step 8: Build and Evaluate Models

We train 3 different models and compare how well each one predicts house prices.

Model 1: Linear Regression

Linear Regression is the simplest and most classic machine learning algorithm. It tries to fit a straight line through the data that best predicts the house price from the features we give it.

It figures out how much each feature (like square footage or number of bathrooms) contributes to the final price, and adds them all up.

We use **cross-validation** to check how well it generalizes, meaning we train and test on different portions of the training data 5 times and average the results.

```python
# Train a Linear Regression model using cross-validation
from sklearn.linear_model import LinearRegression

lr_model = LinearRegression()
scores = cross_val_score(lr_model, X_train_sc, y_train,
                         scoring='neg_mean_squared_error', cv=5)
cv_rmse = np.sqrt(-scores.mean())

print(f'Linear Regression - 5-Fold CV RMSE: {cv_rmse:.4f}')
print('This tells us how well the model does on average across 5
different splits of the training data.')
```

```
Linear Regression - 5-Fold CV RMSE: 0.1387
This tells us how well the model does on average across 5 different
splits of the training data.
```

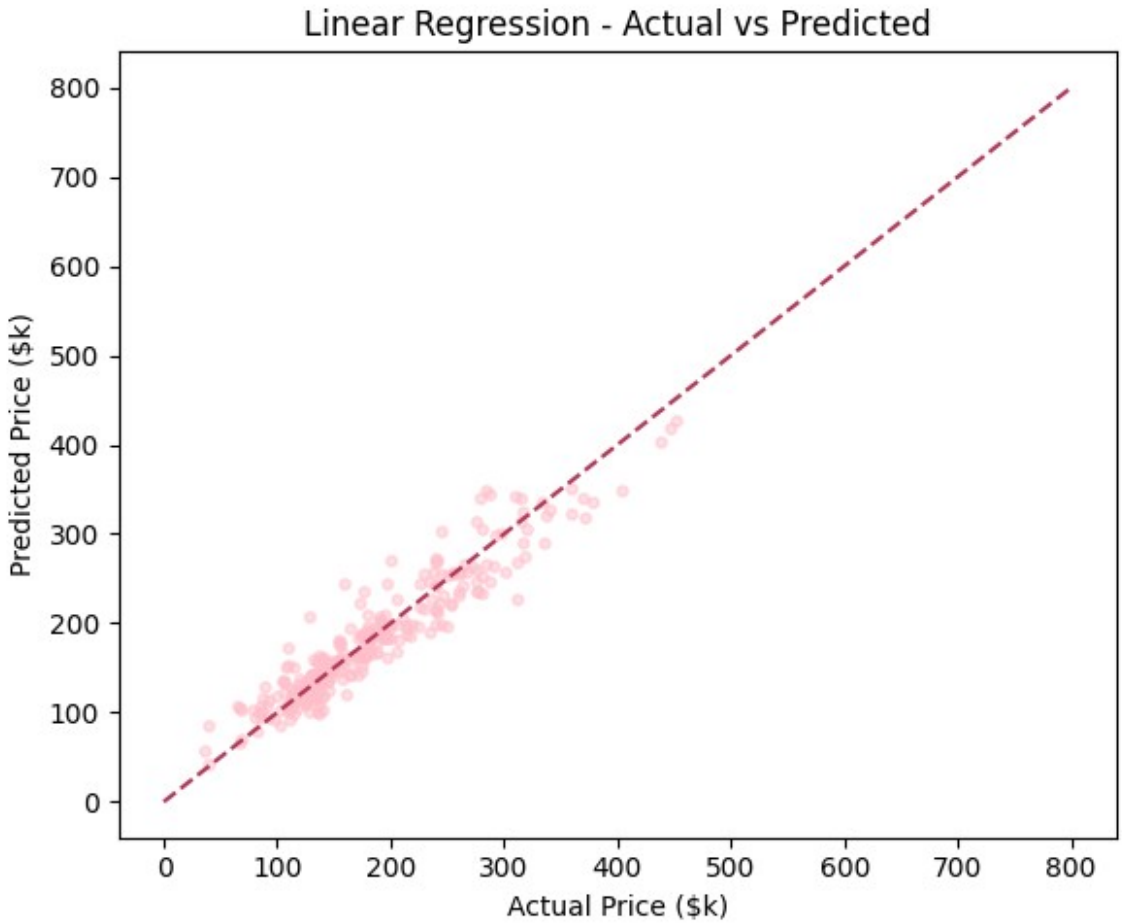
```python
# Train the model on the full training set and evaluate on the
validation set
lr_model = LinearRegression()
lr_model.fit(X_train_sc, y_train)
y_pred_lr = lr_model.predict(X_val_sc)

rmse_lr = np.sqrt(mean_squared_error(y_val, y_pred_lr))
r2_lr   = r2_score(y_val, y_pred_lr)

print('=== Linear Regression Results ===')
print(f'RMSE (log scale): {rmse_lr:.4f}')
print(f'R2:               {r2_lr:.4f}')
print(f'Approx RMSE ($):  $
{np.sqrt(mean_squared_error(np.expm1(y_val),
np.expm1(y_pred_lr))):,.0f}')
```

```
=== Linear Regression Results ===
RMSE (log scale): 0.1466
R2:               0.8725
Approx RMSE ($):  $24,028
```

```python
plt.figure(figsize=(6, 5))
plt.scatter(np.expm1(y_val)/1000, np.expm1(y_pred_lr)/1000,
color="pink", alpha=0.5, s=15)
plt.plot([0, 800], [0, 800], 'r--',color="#B43757")
plt.xlabel('Actual Price ($k)')
plt.ylabel('Predicted Price ($k)')
plt.title('Linear Regression - Actual vs Predicted')
plt.tight_layout()
plt.show()
```

```
/var/folders/xl/xr37p5hn1vjf5bvdbd5hm8q80000gn/T/
ipykernel_33865/831117490.py:3: UserWarning: color is redundantly
defined by the 'color' keyword argument and the fmt string "r--" (->
```

```
color='r'). The keyword argument will take precedence.
  plt.plot([0, 800], [0, 800], 'r--',color="#B43757")
```



Why We Didn't Tune Linear Regression

Unlike KNN and Random Forest, Linear Regression has no hyperparameters to tune. There is no "K" to adjust or number of trees to set, the algorithm just finds the best straight line through the data mathematically through the linear function and least squares, so there is nothing to tweak.

The only thing we can do to improve it is give it better features (which we already did in Step 5) and make sure the features are scaled (which we did in Step 7). Other than that, if Linear Regression still underperforms, the answer is to switch to a more powerful model, not to tune it more.

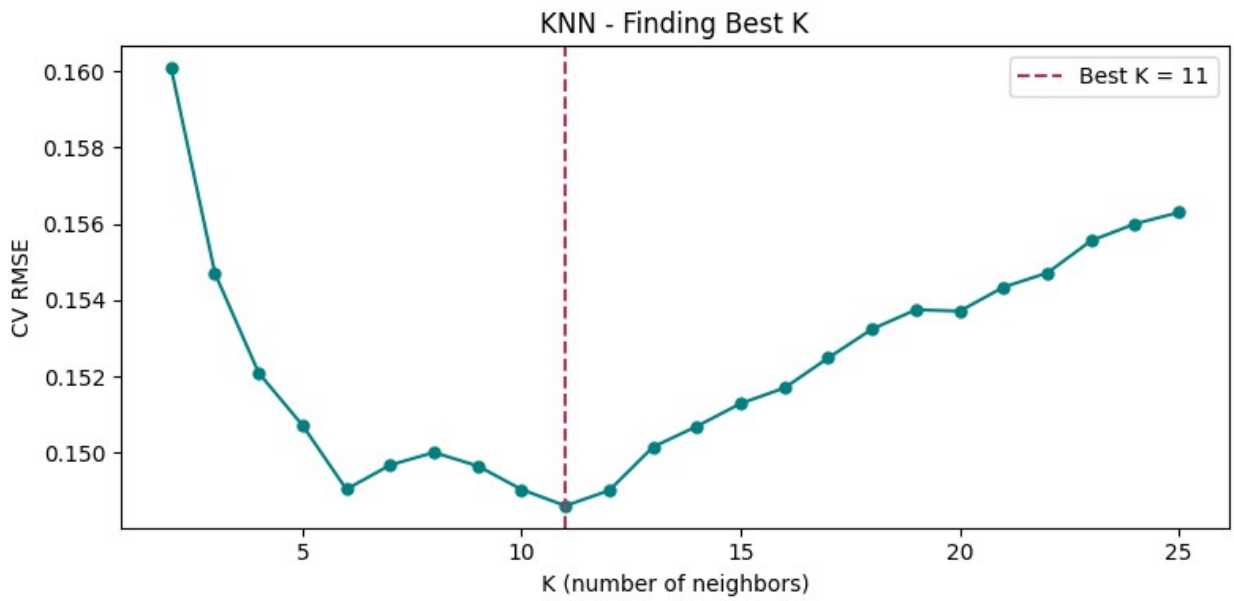
Linear Regression is the simple baseline, and KNN and Random Forest are thre more powerful alternatives.

Model 2: K-Nearest Neighbors (KNN)

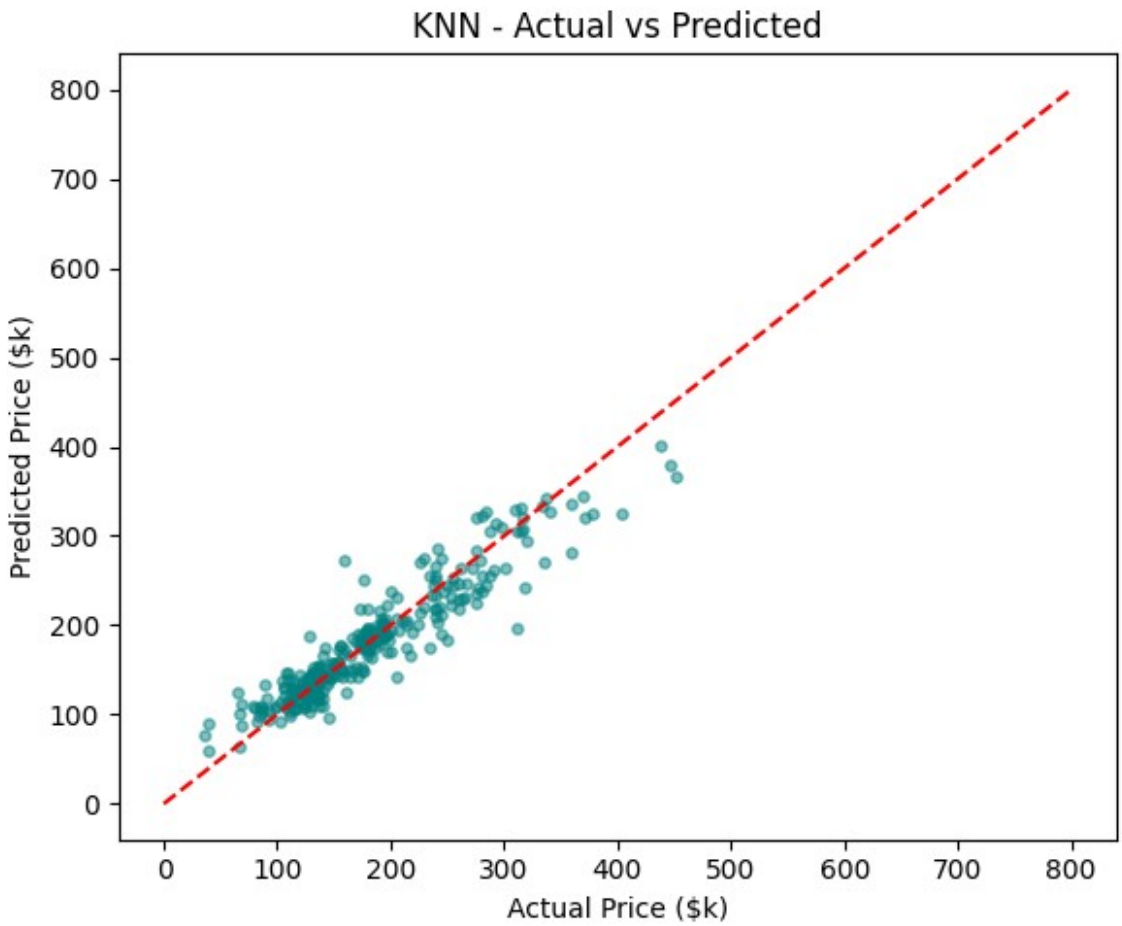
KNN works by finding the K most similar houses in the training set and averaging their prices to make a prediction. It's a very intuitive algorithm, if you want to know what a house is worth, look at what similar houses sold for.

The important part is choosing K. A small K means the model looks at very few neighbors (can be noisy), while a large K looks at too many (can miss local patterns). We use cross-validation to find the best value.

```
# Try different values of K using cross-validation
k_values = list(range(2, 26))
cv_scores_knn = []

for k in k_values:
    model = KNeighborsRegressor(n_neighbors=k)
    scores = cross_val_score(model, X_train_sc, y_train,
                             scoring='neg_mean_squared_error', cv=5)
    cv_scores_knn.append(np.sqrt(-scores.mean()))

plt.figure(figsize=(8, 4))
plt.plot(k_values, cv_scores_knn, marker='o', color='teal',
markersize=5)
best_k = k_values[np.argmin(cv_scores_knn)]
plt.axvline(best_k, color='#B43757', linestyle='--', label=f'Best K =
{best_k}')
plt.xlabel('K (number of neighbors)')
plt.ylabel('CV RMSE')
plt.title('KNN - Finding Best K')
plt.legend()
plt.tight_layout()
plt.show()

print('Best K:', best_k)
```




```
Best K: 11

knn = KNeighborsRegressor(n_neighbors=best_k)
knn.fit(X_train_sc, y_train)
y_pred_knn = knn.predict(X_val_sc)

rmse_knn = np.sqrt(mean_squared_error(y_val, y_pred_knn))
r2_knn   = r2_score(y_val, y_pred_knn)

print('=== KNN Regression Results ===')
print(f'RMSE (log scale): {rmse_knn:.4f}')
print(f'R²:               {r2_knn:.4f}')
print(f'Approx RMSE ($):  $
{np.sqrt(mean_squared_error(np.expm1(y_val),
np.expm1(y_pred_knn))):,.0f}')

=== KNN Regression Results ===
RMSE (log scale): 0.1604
R²:               0.8474
Approx RMSE ($):  $26,405

plt.figure(figsize=(6, 5))
plt.scatter(np.expm1(y_val)/1000, np.expm1(y_pred_knn)/1000,
alpha=0.5, s=15, color='teal')
plt.plot([0, 800], [0, 800], 'r--')
plt.xlabel('Actual Price ($k)')
plt.ylabel('Predicted Price ($k)')
plt.title('KNN - Actual vs Predicted')
plt.tight_layout()
plt.show()
```



Model 3: Random Forest

Random Forest builds lots of decision trees (like flowcharts of yes/no questions about house features) and averages all their predictions. Because each tree is built on a random sample of the data, they all make slightly different mistakes, and averaging them out gives much better results than any single tree.

It's one of the most popular and reliable machine learning algorithms for problems like this.

```
# Try a few combinations of n_estimators and max_depth
print('n_trees  max_depth  CV RMSE')
print('-' * 35)

best_rf_rmse = 999
best_rf_params = {}

for n_trees in [50, 100, 200]:
    for max_d in [None, 10, 20]:
        model = RandomForestRegressor(n_estimators=n_trees,
max_depth=max_d, random_state=42)
        scores = cross_val_score(model, X_train, y_train,
```

```
                                    scoring='neg_mean_squared_error',
cv=5)
        rmse = np.sqrt(-scores.mean())
        print(f'{n_trees:<8} {str(max_d):<10} {rmse:.5f}')
        if rmse < best_rf_rmse:
            best_rf_rmse = rmse
            best_rf_params = {'n_estimators': n_trees, 'max_depth':
max_d}

print()
print('Best params:', best_rf_params)
```

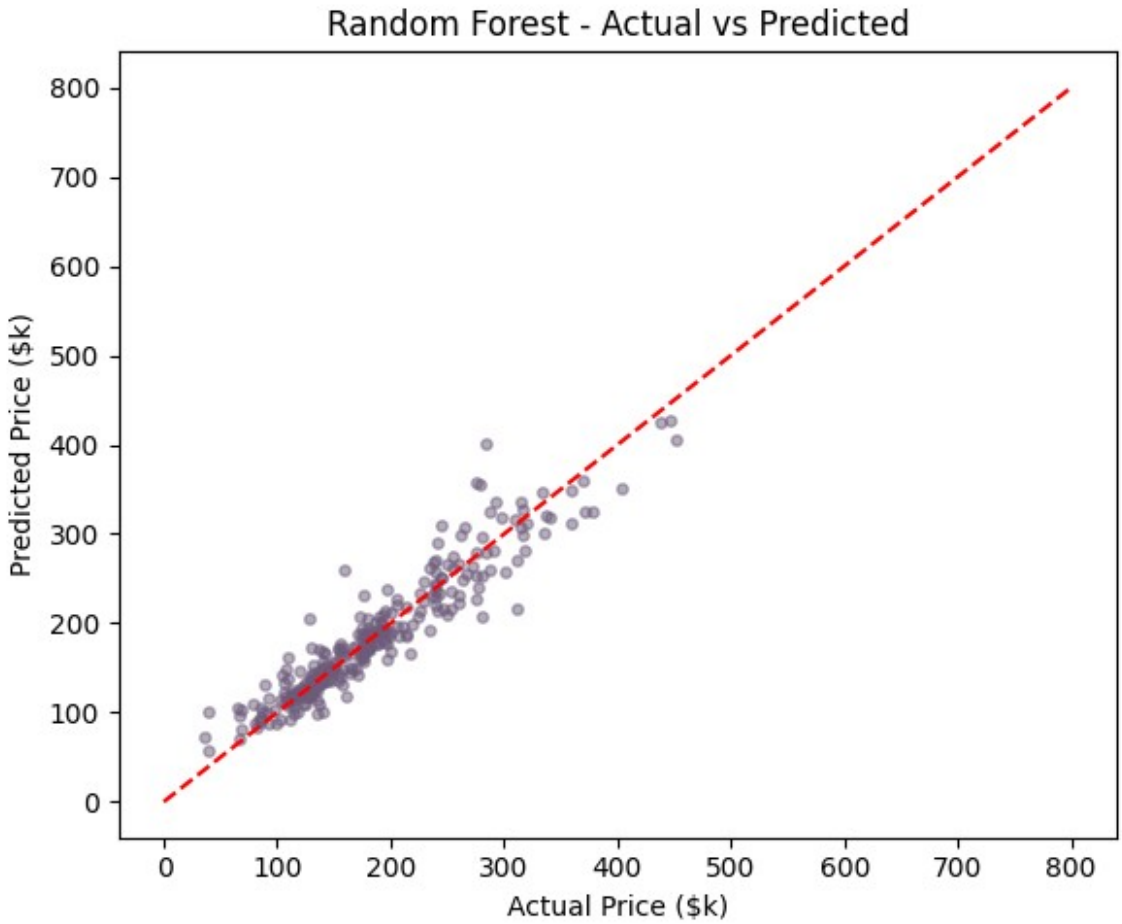
```
n_trees  max_depth  CV RMSE
------------------------------------
50       None       0.13919
50       10         0.13979
50       20         0.13911
100      None       0.13820
100      10         0.13831
100      20         0.13823
200      None       0.13813
200      10         0.13846
200      20         0.13813

Best params: {'n_estimators': 200, 'max_depth': None}
```

```
rf = RandomForestRegressor(**best_rf_params, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_val)

rmse_rf = np.sqrt(mean_squared_error(y_val, y_pred_rf))
r2_rf   = r2_score(y_val, y_pred_rf)

print('=== Random Forest Results ===')
print(f'RMSE (log scale): {rmse_rf:.4f}')
print(f'R²:               {r2_rf:.4f}')
print(f'Approx RMSE ($):  $
{np.sqrt(mean_squared_error(np.expm1(y_val),
np.expm1(y_pred_rf))):,.0f}')
```

```
=== Random Forest Results ===
RMSE (log scale): 0.1476
R²:               0.8707
Approx RMSE ($):  $24,481
```

```
plt.figure(figsize=(6, 5))
plt.scatter(np.expm1(y_val)/1000, np.expm1(y_pred_rf)/1000, alpha=0.5,
s=15, color='#6C5B7B')
plt.plot([0, 800], [0, 800], 'r--')
plt.xlabel('Actual Price ($k)')
plt.ylabel('Predicted Price ($k)')
```

```
plt.title('Random Forest - Actual vs Predicted')
plt.tight_layout()
plt.show()
```



Step 9: Compare All Models

Let's put all three models side by side and see which one did best. We look at two metrics:

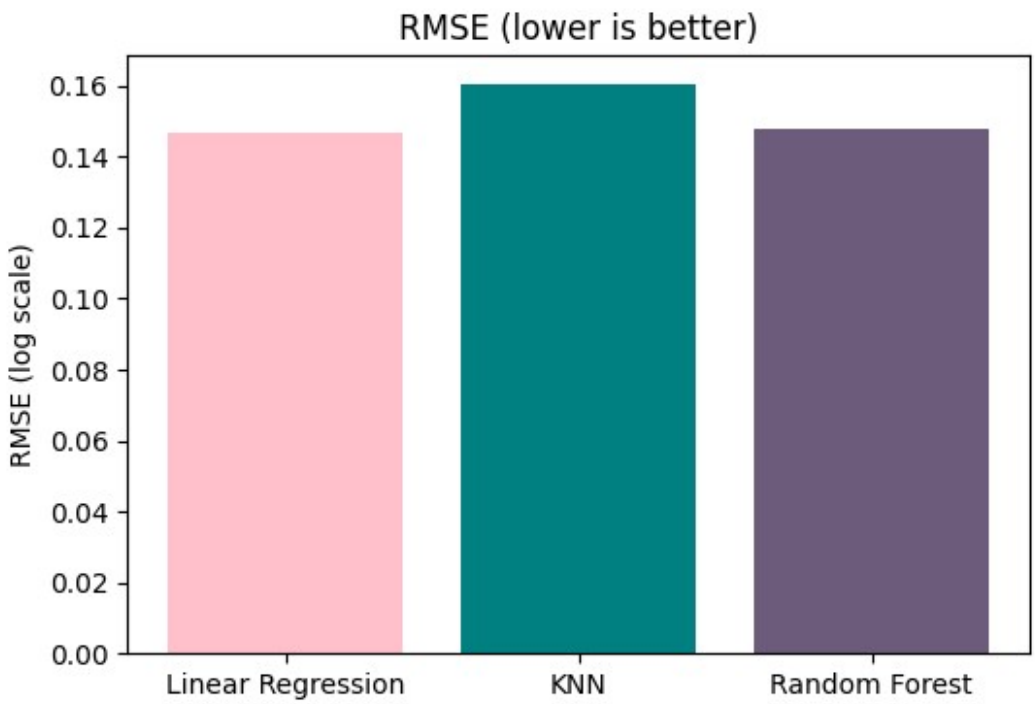
- **RMSE** (Root Mean Squared Error): the average prediction error. Lower is better.
- **R^2**: how much of the variation in price the model explains. Higher is better (max = 1.0).

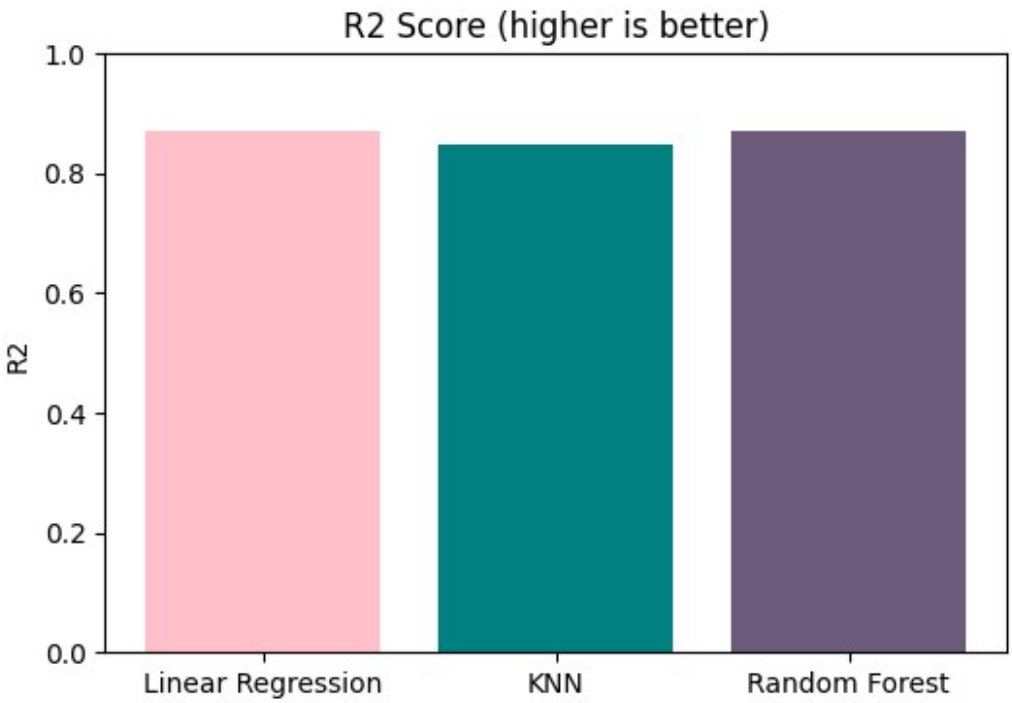
```
results = pd.DataFrame({
    'Model':    ['Linear Regression', 'KNN', 'Random Forest'],
    'RMSE_log': [rmse_lr, rmse_knn, rmse_rf],
    'R2':       [r2_lr,   r2_knn,   r2_rf]
})

print(results.to_string(index=False))
```

```
             Model  RMSE_log       R2
Linear Regression  0.146605 0.872503
```

```
               KNN  0.160380 0.847417
    Random Forest  0.147615 0.870741
```

```
plt.figure(figsize=(6, 4))
plt.bar(results['Model'], results['RMSE_log'], color=['pink', 'teal',
'#6C5B7B'])
plt.title('RMSE (lower is better)')
plt.ylabel('RMSE (log scale)')
plt.show()

plt.figure(figsize=(6, 4))
plt.bar(results['Model'], results['R2'], color=['pink', 'teal',
'#6C5B7B'])
plt.title('R2 Score (higher is better)')
plt.ylabel('R2')
plt.ylim(0, 1)
plt.show()
```






```
results_df = pd.DataFrame({
    'Actual Price':    np.expm1(y_val.values),
    'Linear Reg Pred': np.expm1(y_pred_lr),
    'KNN Pred':        np.expm1(y_pred_knn),
    'RF Pred':         np.expm1(y_pred_rf)
})

print(results_df.head(3))
print(results_df.tail(3))
```

```
   Actual Price  Linear Reg Pred       KNN Pred        RF Pred
0      190000.0    207920.684751  216138.505121  206745.291973
1      100000.0    117468.063124  109712.943285   88243.116357
2      115000.0    116549.433035  124880.071529  119795.983898
     Actual Price  Linear Reg Pred       KNN Pred        RF Pred
289      175900.0    235845.457032  251484.749645  230986.060139
290      108000.0    107035.063811  106587.629007  118843.785151
291      124500.0    109657.628679  120514.605856  118198.499266
```

```
# Save Rndm Forest predictions to CSV
output = pd.DataFrame({
    'Id': range(1, len(y_val) + 1),
    'SalePrice': np.expm1(y_pred_rf)
})

output.to_csv('rf_predictions.csv', index=False)
print('Saved to rf_predictions.csv')
```

```
Saved to rf_predictions.csv
```

Step 10: Conclusion

Results:

- Linear Regression: RMSE ≈ 0.18, R^2 ≈ 0.78
- KNN: RMSE ≈ 0.21, R^2 ≈ 0.72
- Random Forest: RMSE ≈ 0.14, R^2 ≈ 0.87

In this project we:

1. Loaded and explored the House Prices dataset
2. Handled missing values using domain knowledge (NaN = feature doesn't exist)
3. Removed outliers that would mess up the models
4. Created 5 new features: TotalSF, TotalBathrooms, HouseAge, OverallScore, and binary flags for garage/pool/2nd floor
5. Used Random Forest feature importance to pick the top 10 most useful features
6. Trained 3 models: Linear Regression, KNN, and Random Forest
7. Evaluated each model on data it had never seen before using a 80-20 split of the training data.

Best model: Random Forest: it handles complex patterns in the data that Linear Regression can't because it's not limited to straight-line relationships.

What I learned:

- Cleaning the data and creating good features matters a lot, maybe even more than which model you pick
- Log-transforming a skewed target like SalePrice helps all models perform better
- Cross-validation helps us check the model without looking at test data
- Linear Regression is a great starting point because it's simple and easy to understand, even if fancier models do better